

SYNOPSIS

Pricing the Probe: When a Web-Attack Detector Should Deceive Rather Than Decide

*An Active Deception Framework for Web Attack Detection
using Response-Side Probes and State-Consistent Decoys*

TABLE OF CONTENTS

1. Introduction.....	4
1.1 Problem Definition.....	4
1.2 Problem Overview	4
1.3 Hardware Specification.....	7
1.4 Software Specification	8
2. Literature Review.....	10
2.1 Existing System	10
2.2 Proposed Systems	12
2.3 Literature Review Summary	15
3. Project Formulation	18
3.1 The setting.....	18
3.2 The cost table	18
3.3 The negative result that motivates everything	19
3.4 What a probe is worth	19
3.5 The bands that come out	20
4. Research Objectives.....	22
5. Methodologies.....	24
5.1 Step 1: working out which session a request belongs to.....	24
5.2 Step 2: turning a request into numbers	24
5.3 Step 3: the dual suspicion meter	25
5.4 Step 4: the decision policy	26
5.5 Step 5: the bait library and its calibration	27
5.6 Step 6: the invisibility gate	29
5.7 Step 7: spotting a bite and updating the belief.....	31
5.8 Step 8: the decoy and its fact store.....	31

5.9 Step 9: tamper-evident logging.....	32
6. Experimental Setup.....	33
6.1 The target application and the stack.....	33
6.2 Building the corpus.....	33
6.3 A benign corpus built to be hard.....	34
6.4 Arms, rounds and the randomised holdout.....	35
6.5 Metrics and statistical protocol.....	36
6.6 Results obtained so far.....	38
6.7 Implementation status.....	43
7. Conclusion.....	44
7.1 Limitations.....	44
7.2 What remains.....	45
8. Tentative Chapter Plan for the Proposed Work.....	46
9. References.....	47

1. INTRODUCTION

1.1 Problem Definition

A security layer that sits in front of a website has only two things it can do with an incoming request. It can let the request through, or it can block it. Both of these are reactions. The layer can only judge what the visitor has already sent, so the quality of every decision depends on how much the visitor has already given away.

This creates a problem with no comfortable middle. If the defence acts early, while the evidence is still thin, it will block ordinary people who have done nothing wrong. Blocking a real customer is expensive, and the customer notices. If the defence waits until the evidence is strong, the attacker has already had ten or twenty requests to work with, and by then the damage may already be done. Signature-based firewalls sit at one end of this problem and machine-learning detectors sit at the other. Neither of them escapes it, because both only read traffic that somebody else chose to send.

Deception is the usual answer to this passivity, and it genuinely works. The difficulty is where people put it. A honeypot is a separate machine on a separate address, so it only sees the attackers who happen to wander onto it. An attacker who is aiming at the real website never touches one. A honeytoken does sit inside the real system, but it is planted once and shown to everybody, so it cannot be a reaction to one particular visitor. Neither of these helps a detector that is unsure about the person it is talking to at this moment.

So the question this project asks is narrower than “how should a website deceive an attacker”. It is a question that can actually be answered with a number:

For a detector that today has only two actions, is there a third action that is worth taking while it is still unsure, and can the right moment to take it be worked out from something solid instead of being chosen by hand?

1.2 Problem Overview

1.2.1 What the third action is

The third action we propose is a response-side probe. When the system is unsure about a visitor, it adds something small to the reply that the browser will never show on screen, but that anyone reading the raw HTTP reply will spot immediately. Three examples from the library we built:

- **A fake error.** a fake database error message that names a table which does not exist anywhere in the real system;
- **An unused field.** an extra field added to a JSON reply that no part of the real website ever reads or writes;
- **A hint at a hidden page.** an HTML comment that mentions an old login address which was supposedly left behind by mistake.

An ordinary user sees none of this. Their page looks exactly the same as it always did. We do not simply assume that; Section 5.6 explains how each probe is tested and how the ones that fail are thrown away.

Someone who is probing the site behaves differently. They read the raw reply, they notice the hint, and often enough to be useful, they try to use it. The moment they do that, they have created evidence about themselves that no amount of patient watching would ever have produced. They are still not blocked. The session is quietly moved to a fake copy of the website. From there on, everything they do is recorded and nothing they take away is real. They are never shown an error, so they have no reason to think anything went wrong.

1.2.2 Why the timing is the real research problem

Planting a probe is easy. Deciding when to plant it is the hard part, and it is the part this project is really about. If the answer is “when the suspicion score goes above 0.4”, then the whole method is only as good as whoever picked 0.4, and an examiner is right to be unimpressed. So we do not pick a number. We treat the probe as buying information, work out what that information is worth, and let the arithmetic decide.

One result is worth stating right at the start, because it is the part that surprised us while building the system. If you only count costs, probing is *never* the cheapest thing to do. At every level of suspicion short of complete certainty, probing costs strictly more than simply letting the request through. The rule therefore collapses back into an ordinary two-way choice with a single cut-off at 0.816. In other words, **there is no middle band to tune, because on cost alone there is no middle band at all**. The band only appears once we calculate what the information is worth and subtract it. It also closes again at both ends, because when you are already sure, an extra observation cannot change your mind and is therefore worth nothing. This makes the trigger point a consequence of the

numbers rather than a setting, which means there is no threshold for an attacker to find and stay under.

1.2.3 What the system looks like

The system has seven parts, and only one of them contains anything that was learned from data. Every request goes through a reverse proxy. The proxy works out which session the request belongs to, forwards it to the real application first, and only then scores it. Forwarding first is deliberate: if any part of the detection layer crashes, the user has already been served, so the security layer can never take the website down. After forwarding, the proxy turns everything the session has done so far into eighteen numbers, updates two running scores, prices the three possible actions, and picks the cheapest. Figure 1 shows the whole path.

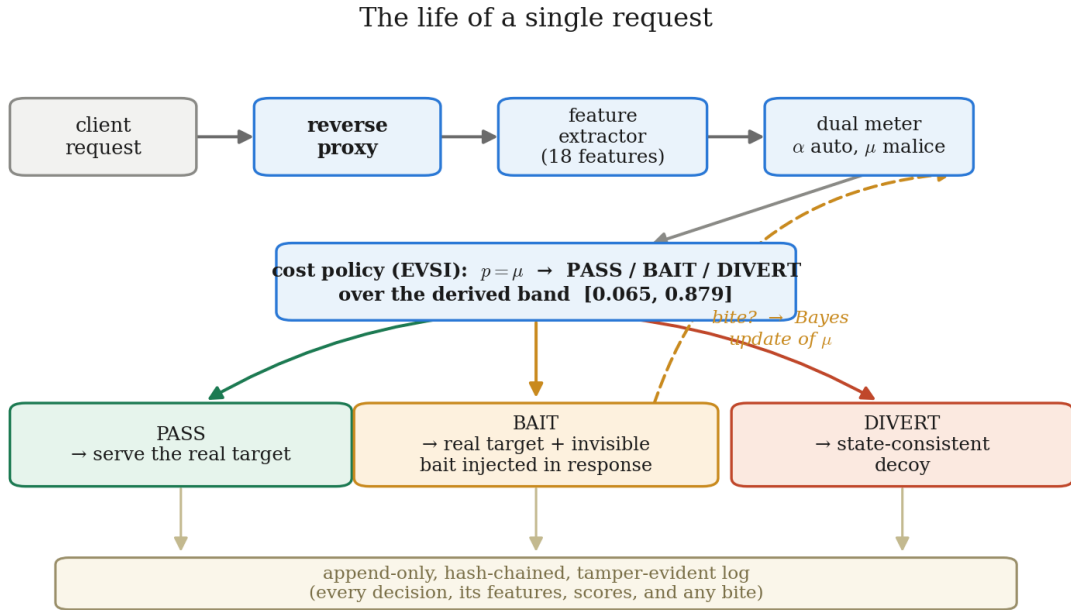


Figure 1. The path of a single request. The proxy forwards the request first and scores it afterwards, so a failure in the detection layer can never affect the user. Every decision, along with the features and scores behind it, is written to a log where each entry contains a hash of the one before it, so any later tampering can be detected.

The three outcomes are simple to describe. PASS serves the ordinary reply and changes nothing. BAIT serves the ordinary reply with an invisible probe added to it. DIVERT sends this request, and every later request from the same session, to the decoy copy of the site.

1.2.4 A worked example

Table 1 follows one careful human attacker who is probing the search box by hand. It shows how suspicion builds up, when the probe is planted, and what happens when the attacker uses it.

Table 1. *One session from start to diversion. The band edges, the cost arithmetic and the likelihood ratio are the real values used by the system.*

Req	What the visitor does	What the system notices	Suspicion p	Action
1	Loads the home page	Images and stylesheets fetched, normal browser headers	0.01	PASS (below 0.0647)
2	Searches for “laptop”	Ordinary input, nothing unusual	0.02	PASS
3	Searches for “laptop” (with an apostrophe)	Special-character ratio rises, the server returns an error 500	0.09	BAIT** — injection suspected, so the fake database error is chosen
—	The reply now carries a fake error naming a table called acct_shadow_a3f9. On screen the page looks like any other error page.			
4	Searches for “SELECT * FROM acct_shadow_a3f9”	BITE.** The visitor has used a table name that exists nowhere except in our fake error	0.99	DIVERT
5+	Continues attacking	Every request now goes to the decoy	—	Recorded, never blocked

The point of the example is the size of the jump at request 4. Without the probe, the passive score after four requests was only about 0.12, and it would have needed many more requests to climb past 0.86 on its own. One planted probe and one bite got there in a single step. Section 6.4 explains how we measured that saving in a way that proves the probe caused it, rather than just observing that it happened.

1.3 Hardware Specification

The whole system is meant to run on one ordinary machine with no outside service of any kind, and it does. Table 2 lists the machine used for development and for every number reported in this document.

Table 2. *Host machine used for development and evaluation.*

Component	Specification	What it is used for
Processor	Intel Core Ultra 7 155H, 16 cores / 22 threads	Runs the real application, the proxy, the decoy and all the traffic generators at the same time
Memory	32 GB	Building the corpus and running the 100-seed evaluation sweep are the heaviest steps
Graphics	Intel integrated graphics; NVIDIA	Not used by the detector at all. Used only for the optional local language-model test in

Component	Specification	What it is used for
	RTX A500 Laptop GPU (4 GB)	Section 6.6
Storage	400 GB NVMe SSD partition, of which about 2 GB is used	Traffic logs, the hash-chained record store, and the frozen model files
Network	Loopback only (127.0.0.1)	Nothing in the stack is ever attached to an address that another machine can reach

None of this is a requirement. The only part of the system that learns anything is a pair of logistic regressions fitted to a few thousand sessions. That is a deliberate choice, not a compromise. At this sample size a deep neural network would be harder to defend in an examination than it would be to train, and it would hide the reasoning behind each decision. A four-core machine with 8 GB of memory will reproduce every result in this document, just more slowly.

There is one genuine hardware constraint, and it is about safety rather than speed. The target application is deliberately full of security holes, because it has to be for any of this to be testable. It must therefore stay on an isolated machine and must never be attached to a network address that anyone else can reach.

1.4 Software Specification

Everything is written in one language, which keeps the proxy, the traffic generators, the analysis and the tests consistent with each other. Table 3 lists each layer with the exact version that produced the results.

Table 3. *Software stack, with the version every result was produced under.*

Layer	Software and version	Why this choice
Operating system	Windows 11 Pro, build 26200, 64-bit	The development host. The stack is pure Python, so it also runs unchanged inside Linux containers
Language	Python 3.11	One language across the proxy, the generators, the analysis and the test suite
Web layer	FastAPI 0.115.6, Uvicorn 0.34.0, Jinja2 3.1.5	Handles many requests at once in the proxy, and renders ordinary HTML pages in the target application
HTTP client	httpx 0.28.1	Used by the proxy to forward requests upstream, and by every traffic generator
Database	SQLite by default; PostgreSQL 16 through psycopg 3.2.3 under Docker	PostgreSQL is used where the exact wording of a real database error matters, because our fake errors have to look like the real ones
Machine learning	scikit-learn 1.6.1, NumPy 2.2.1, pandas 2.2.3, joblib 1.4.2	Two small logistic models, chosen so that every decision can be explained by looking at the feature weights
Response analysis	BeautifulSoup4 4.12.3	Compares what a browser would actually paint on screen, before and after a probe is

Layer	Software and version	Why this choice
		added
Figures and analysis	matplotlib 3.10.0	Every figure in this document is drawn from the frozen model and the real result files, not sketched by hand
Testing	pytest 8.3.4, pytest-asyncio 0.25.2	285 tests, including the ones that stop the cost table or the record format from being changed quietly
Containers	Docker and Docker Compose	So the target application and its database can be reproduced on a machine that is not the author's
Optional component	Ollama running llama3.2:1b, loopback only	Used once only, to show that the decoy's consistency does not depend on which generator produces the fake content
Configuration	PyYAML 6.0.2	Holds the cost table and the system settings. Both files are hashed and checked at start-up

2. LITERATURE REVIEW

This review is built on thirty-eight primary references covering cyber deception, honeypots, honeytokens, web-attack detection, decision theory and the methodology of evaluating security systems. Each one was opened on its publisher page and checked for title, author order, venue, year and DOI, rather than copied from a citation manager. A further set of recent papers on machine-learning detection of individual vulnerability classes supports the detection tier.

The search covered the ACM Digital Library, IEEE Xplore, USENIX, SpringerLink, arXiv and DBLP. Priority went to work published from 2020 onward, and seventeen of the thirty-eight, or 45%, are from that period. Older papers were kept only where they are the original source of an idea this project depends on, such as Howard’s value of information [29], Elkan’s cost-sensitive learning [30] and Axelsson’s work on base rates [32]. Papers with no measurement in them were left out, as were papers on network-level or IoT honeypots with nothing to say about the application layer.

2.1 Existing System

What is actually running in front of real websites today falls into three families. Each one is passive, but for a different reason, and it is worth being precise about what each one can and cannot do.

2.1.1 Rule-based web application firewalls

These match the text of a request against a list of known-bad patterns. Kruegel and Vigna [22] provided the foundation by learning, from normal traffic, what each parameter of each page usually looks like in terms of length, character mix and structure, and then flagging anything that departs from it. Robertson and colleagues [23] extended this by grouping anomalies into explainable classes, so that the system could say what kind of problem it had found rather than just that something was odd.

The weakness here is not a lack of cleverness. Amouei and colleagues [26] used reinforcement learning to search automatically for payloads that slip past well-configured firewalls, and beat the previous best bypass methods by 33.53% on average. Our own measurements agree. In our evaluation the signature layer caught 40.8% of attacks. It handled textbook injection reasonably well and was almost blind to everything else, for a

simple reason: an attack that just changes an identifier in a web address is perfectly valid text, so there is no bad pattern to match against.

2.1.2 Machine-learned request classifiers

Tekerek [25] applies a convolutional neural network to HTTP anomaly detection and is a fair representative of where this line of work has gone: better classifiers, still entirely passive. The field’s standard benchmark, the CSIC 2010 dataset [24], contains about 36,000 normal and over 25,000 anomalous requests. It has no sessions, no timing information, no benign but automated traffic, and no bait interactions of any kind. That is why we could not use it: not one of the numbers this project reports can be computed from it.

Iliou and colleagues [27, 28] explain why a second axis of measurement is needed at all. Advanced bots now imitate human browser fingerprints and human-like behaviour closely enough that automation signals on their own stop separating them from people. That finding is the direct reason our system keeps two separate scores instead of one, as Section 5.3 describes.

2.1.3 Honeypots

Provos’s Honeyd [6] set the classical pattern: many virtual machines simulated on unused addresses. That is exactly the placement we are trying to avoid, because a honeypot on a separate address can only catch someone who goes looking for it. Nawrocki and colleagues [7] catalogue the available software and fix the low, medium and high interaction vocabulary that the field still uses. Javadpour and colleagues [8] survey recent efforts to make honeypots more convincing, which tells us something in itself: the field’s attention has been on realism rather than on what deception costs honest users.

Two measurement papers matter more to us than the surveys. Barron and Nikiforakis [9] ran 102 medium-interaction honeypots for four months, varying the location, the difficulty of breaking in, and the files they planted, and found that what you plant measurably changes how attackers behave. That is direct support for choosing bait per session instead of using the same bait for everyone. Vetterl and Clayton [10] fingerprinted honeypots across the whole internet using a single packet, reached an equal error rate of 0.0183, and located 7,605 live honeypot instances across nine implementations. Their result is the strongest argument we know of for treating invisibility as something that must be measured and certified rather than assumed.

2.1.4 Honeytokens and decoys

Yuill and colleagues [14] introduced the honeyfile, a bait file on a shared drive whose opening raises an alarm. Bowen and colleagues [15] generated decoy documents automatically and, more usefully for us, wrote down the four properties a good decoy needs: it must be believable, enticing, non-interfering, and detectable when used. Juels and Rivest [16] gave us honeywords, which are fake passwords stored next to the real one, and which remain the cleanest example of a tripwire whose only job is to produce evidence.

All three share the same limitation. The token is always switched on, it is identical for every visitor, and there is no decision about when to deploy it because it is never not deployed. Srinivasa and colleagues [17] then showed that poorly built tokens can themselves be fingerprinted and avoided, and Timmer and colleagues [18] pointed out that the field measures realism and enticement carefully but does not measure the cost to honest users at all.

Table 4 puts the three families side by side. Reading the last row is the quickest way to see what this project is doing differently: in every existing family the threshold is either chosen by a person or fitted to a validation set, and in ours it falls out of the cost table.

Table 4. *The three deployed defence families, compared with the approach proposed here.*

	Rule-based WAF	ML classifier	Honeypot	This work
Where it sits	In front of the app	In front of the app	Beside the app	In front of, and inside, the app
Where evidence comes from	Request matches a known pattern	Request looks unusual	The attacker walks in	The defender provokes a reaction
When evidence arrives	Immediately, and often wrong	Slowly	Only if the attacker strays	On demand, when the system is unsure
Does the attacker learn they were caught?	Yes, they are blocked	Usually, they are blocked	No	No, the diversion is silent
Cost to honest users	Wrongly blocked users	False alarms	None	Measured, and bounded by the gate
Where the threshold comes from	An administrator picks it	Tuned on a validation set	Not applicable	Derived from the cost table and the information value

2.2 Proposed Systems

Alongside what is deployed, three lines of research have proposed alternatives in the last few years. These are the works this project is closest to, and therefore the ones it has to be clearly distinguished from.

2.2.1 Language-model honeypots

Sladić and colleagues [11] built shellLM, a fake Linux shell that uses a language model to invent command output on demand. Human researchers judging whether each reply was real or fake were convinced 92% of the time, and that experiment set the evaluation style the subfield now uses. Reworr and Volkov [12] went a step further and placed prompt injection inside an SSH honeypot aimed specifically at automated AI attackers, collecting over eight million attempts in about three months. Putting something into the reply in order to provoke a reaction is the closest published idea to our probe, although it works at the shell level rather than inside a web response, and it targets AI agents rather than people.

Bridges and colleagues [13] surveyed this whole area and reported two things we took seriously. Real deployments show only incremental progress, and the field has no agreed way of evaluating these systems at all. The most architecturally advanced recent work in this line is a stateful, multi-agent shell honeypot that keeps a persistent filesystem so that several attackers see the same evolving world. Its evaluation reaches a conclusion that shaped our own design: better generation does not by itself produce better deception, and keeping a story consistent across a long session is the harder and more important problem.

One design consequence follows directly. Because a language model in the reply path adds delay, and a slow fake site is a detectable fake site, we moved all generation offline. Nothing in our request path waits for a model.

2.2.2 Deception inside the application layer

This is the neighbourhood our project occupies, and it is small. Kahlhofer and Rass [19] reviewed it and found only nineteen technical methods in total, reporting in plain terms that everything beyond honeypots and reverse proxies has “received little research interest”. That single sentence both defines the space and says that it is nearly empty, which is why it is the most important related-work citation in this synopsis.

Kahlhofer and colleagues [20] then built Honeyquest, which turns deception techniques into a machine-readable description and measures how tempting each one is by questionnaire, without having to build any of them. They report that the presence of deception cut the risk of an adversary finding a real security weakness by about 22% on average. The method is useful for ranking bait ideas before building them, but a

questionnaire measures what people say they would click, not what they do in a live system. Our bite rates measure the second thing.

Koney [21] is the closest engineering neighbour: a Kubernetes operator that automates setting up, rotating and tearing down honeytokens and fake API endpoints, using eBPF to detect and log access. It solves deployment and lifecycle properly. What it does not do is decide when to deploy based on a belief about the visitor who is present right now, and that gap is precisely where this project sits.

2.2.3 Decision-theoretic and game-theoretic deception

Almeshekah and Spafford [1] gave deception its planning model: decide what you want the attacker to believe, then build the deception backwards from that belief. Han, Kheir and Balzarotti [2] provided the standard taxonomy, splitting deception by what is faked and where it sits, and established that bait embedded in a reply is a different category from network or file decoys. Pawlick, Colbert and Zhu [3] define six families of deception and place work like ours in the honey-x and attacker-engagement corner, while noting that most models there are analytical rather than implemented and measured.

Their later paper [31] extends signalling games with a detector that issues probabilistic warnings when a sender is being deceptive, and finds that a high-quality detector removes some pure-strategy equilibria. Zhu and colleagues [4] survey game-theoretic and machine-learning approaches together and conclude that, in practice, deployment decisions remain mostly heuristic. The newest major survey, by Beltrán López and colleagues [5], confirms as of 2025 that the timing question is still on the field’s list of open challenges.

So the ingredients we need are all present in this literature. What is consistently missing is an answer to when that a running system could actually compute. The answer is either a hand-set threshold or an equilibrium that needs assumptions no live system can check.

2.2.4 What we propose against that background

Our proposal is deliberately narrow. We do not propose a new deception technique or a better content generator. We take one decision, the timing of a single probe inside one request-and-reply cycle, and we price it using Howard’s expected value of sample information [29] on top of Elkan’s cost-sensitive decision framework [30]. The mathematics involved is textbook, and we say so at every point in this document rather

than dressing it up. What is new is the application, and two things that fall out of it: the trigger point stops being something anyone can tune, and the safety of the rule can be shown to hold across the entire range of the one number we had to estimate.

2.3 Literature Review Summary

Reading the three deployed families and the three research lines together, the same picture appears from several directions. Deception is placed almost everywhere except in the reply of the live application. When it is placed there, it is static and identical for everyone. And almost nobody measures what it costs the people who were never the target in the first place. Table 5 makes the placement question concrete.

Table 5. *Where the deception is placed, layer by layer.*

Layer	Refs	What is faked	Reaches attackers on the real site?	Chosen per visitor?
Network or host	6, 7	Whole fake machines	No, it is on a separate address	No
File system	14, 15, 18	Fake documents	Only after a break-in	No
Credential store	16	Fake passwords	Only after the database is stolen	No
Cluster configuration	21	Honeytokens and fake API endpoints	Yes, but placed statically	No
Shell session	11, 12	Generated shell output	Only inside the honeypot	Partly (12)
HTTP reply of the live app	This work	Error text, JSON fields, endpoint hints	Yes, always	Yes, chosen per session

Table 6 asks a harder question of the same literature. For each of the six properties this project claims, which prior works already establish it? We built this table as something an examiner could attack, not as decoration. No row except the last carries more than two full marks.

Table 6. *Gap matrix. Y means established, P means partly, and a dash means not addressed.*

Reference	Live reply	Per-session	Derived trigger	Invisibility	Consistency	Causal effect
[6] Provos (Honeyd)	—	—	—	—	—	—
[9] Barron & Nikiforakis	—	—	—	—	—	P
[10] Vetterl & Clayton	—	—	—	—	P	—
[12] LLM Agent Honeypot	P	—	—	—	—	—
[15] Decoy documents	—	—	—	P	—	—
[17] Honeytoken fingerprinting	—	—	—	P	—	—
[18] Honeyfile metrics	—	—	—	—	P	—
[19] App-layer deception survey	Y	—	—	P	—	—
[20] Honeyquest	Y	—	—	—	—	—
[21] Koney	Y	—	—	—	—	—
[31] Signalling games with	—	P	P	P (theory)	—	—

Reference	Live reply	Per-session	Derived trigger	Invisibility	Consistency	Causal effect
evidence						
[35] Ferguson-Walter et al.	—	—	—	—	—	Y
This work	Y	Y	Y	Y	Y	Y

The columns, read left to right, mean: the deception lives inside the live reply; it is chosen per session from a belief about the visitor; the trigger point is derived rather than tuned; invisibility to honest users is measured; the decoy’s consistency is measured; and the effect of the deception is established causally rather than by comparison.

Six gaps follow from all of this, and each one maps to a specific decision in our design.

1. **Gap 1. Deception sits outside the real request path.** Honeypots are separate systems [6, 7, 8], and decoys live in files [14, 15, 18] or credential stores [16]. An attacker working on the real application is never touched by any of them. Our answer is to inject the probe into the live reply itself.
2. **Gap 2. The decision of when to deceive is not modelled.** Tokens are always on [14, 15, 16, 21]. Where a decision exists at all it is a tuned threshold [22, 23, 25] or a game-theoretic equilibrium requiring strong assumptions [3, 31]. Our answer is a band derived from the cost table and the measured probe effectiveness, which disappears completely if the information term is removed.
3. **Gap 3. The evidence value of a trap is asserted rather than measured.** A bite raises an alarm [14, 16], but nobody reports how far a bite should move a belief, and nobody uses the information contained in a visitor who declines the bait. Our answer is a likelihood ratio for both outcomes, estimated in a dedicated calibration round.
4. **Gap 4. The cost to honest users is not measured.** The field measures realism and enticement [18, 20] and detectability by attackers [10, 17], but not what deception does to people who were never the target. Our answer is an invisibility gate with a stated equivalence bound [38], tested against a benign corpus built to be as hard as we could make it.
5. **Gap 5. Causal claims are almost absent.** Only Ferguson-Walter and colleagues [35] ran a controlled experiment, and it needed expensive human subjects. Every other claim that deception helped compares two different systems, so the effect is tangled up with everything else that differs between them. Our answer is a randomised holdout inside a single running system.
6. **Gap 6. Generated fake worlds contradict themselves and nobody counts it.** Language-model honeypots generate content live [11, 12], the field’s own survey

names evaluation as an open problem [13], and contradiction is exactly how an experienced attacker spots a trap [10]. Our answer is a persistent fact store and a contradiction rate we actually report.

3. PROJECT FORMULATION

This section sets out the problem precisely, because the central claim of the project is a claim about a decision rule, and a claim like that has to be checkable by someone who did not write the code.

3.1 The setting

A session is a series of requests that belong to the same visitor. After each request the system holds a number p between 0 and 1, which is how strongly it currently believes the session is hostile. Three actions are available:

- **PASS** — forward the reply exactly as the application produced it.
- **BAIT** — forward the reply with an invisible probe added to it.
- **DIVERT** — send this request, and every later request from this session, to the decoy.

Every combination of an action and the truth carries a cost. Those costs are written down before any evaluation is run, the file is hashed, and the system refuses to start if the numbers have changed. That check exists for a specific reason. The thresholds are derived from these numbers, so editing them after seeing the results would silently invalidate every comparison in the project.

3.2 The cost table

Table 7. *The frozen cost table, and the reasoning behind every cell.*

Situation	Cost	Reasoning
Benign visitor, PASS	0.0	The correct outcome. This anchors the whole scale at zero
Benign visitor, BAIT	1.0	An ordinary user received a probe and never noticed it. We deliberately did not put zero here. Zero would claim the invisibility gate is perfect. This small residual prices the risk that a faulty probe slips past the gate, and it stops the policy from treating probing as free
Benign visitor, DIVERT	200.0	A real user has been sent into the fake site. This is the outcome that must almost never happen, so it is priced at 200 times a wasted probe. The policy therefore demands overwhelming evidence before it will do this
Attacker, PASS	25.0	A missed attacker reached the real application without being observed
Attacker, BAIT	25.0	Deliberately the same as PASS, because the request still reaches the real application. Probing an attacker costs exactly what letting them through costs
Attacker, DIVERT	−20.0	The win. The attacker is contained in the decoy, learns nothing real, and generates intelligence for us. A negative number means the outcome is desirable

The most important cell is the one for probing an attacker. In our first version it was 8.0, following the original specification, which said to discount it because information had

been purchased. That turned out to be double counting. The value of the information is calculated explicitly and subtracted at decision time, so a cost table that also discounts in advance counts the same benefit twice. The probing band would then be an artefact of an arbitrary constant rather than a real result. Probing now carries its full cost. It wins on information value alone, or it does not win at all.

3.3 The negative result that motivates everything

The expected cost of an action at belief p is just the cost when the visitor is an attacker, weighted by p , plus the cost when the visitor is benign, weighted by the remaining probability. Putting the frozen numbers in gives three straight lines:

$$E[PASS] = 25p \quad E[BAIT] = 1 + 24p \quad E[DIVERT] = 200 - 220p$$

Subtract the first from the second and something awkward appears. The difference is $1 - p$, which is positive for every p below 1. In plain terms, probing is strictly more expensive than simply passing the request through, at every level of suspicion short of complete certainty. So on cost alone the rule reduces to a two-way choice, and the boundary sits where passing and diverting cost the same:

$$25p = 200 - 220p \Rightarrow 245p = 200 \Rightarrow p^* = 200/245 = 0.8163$$

This is the sharpest available answer to the obvious objection, which is that our third action is just a threshold we tuned. Without an information term there is no third action to tune, because there is no middle band anywhere on the scale. A unit test in the project enforces this, so it cannot quietly stop being true.

3.4 What a probe is worth

A probe earns its place by buying information. It is described by two numbers, both measured rather than assumed: how often an attacker bites, and how often a benign visitor bites. If we see a bite, Bayes' rule tells us how far our belief should move upward. If we see the probe declined, the same rule tells us how far it should move downward. The expected value of sample information, written $V(p)$, is simply how much cheaper our best decision becomes, on average, once we know which of those two things happened:

$$V(p) = \text{best cost now} - [P(\text{bite}) \times \text{best cost after a bite} \\ + P(\text{no bite}) \times \text{best cost after no bite}]$$

Only the probing action gets this value subtracted, because passing and diverting end the decision and buy nothing. Whichever of the three effective costs is lowest is the action taken.

Two properties matter, and both are enforced as tests rather than trusted. First, $V(p)$ is never negative, because a minimum over straight lines is concave and Jensen's inequality applies. In words: information never hurts you. Second, $V(0)$ and $V(1)$ are both exactly zero, because when you are already certain, no observation can change your decision, so paying for one is pointless. That second property is what closes the probing band at both ends automatically. The band cannot be widened by tuning, and it cannot swallow the whole scale.

We want to be exact about credit here. The expected value of sample information is Howard's [29], and the non-negativity result is a standard lemma. We claim the application of it to this decision, and nothing more than that.

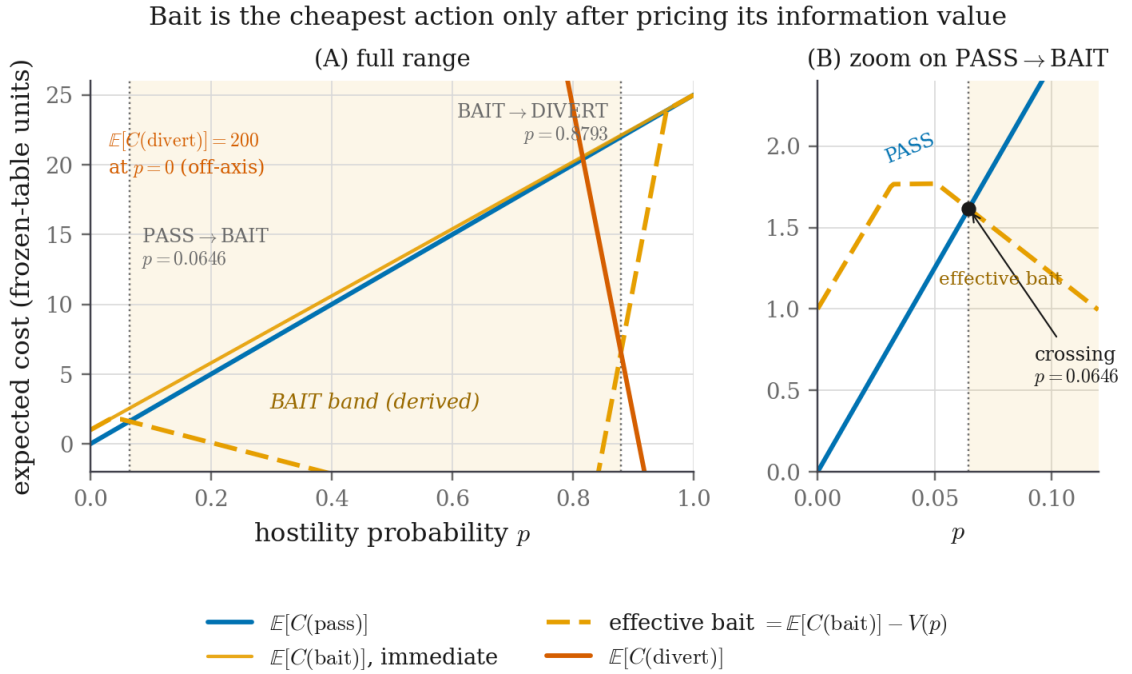


Figure 2. The expected cost of each action plotted against belief, with the information value $V(p)$ drawn beneath. $V(p)$ starts at zero, rises to a peak where uncertainty is greatest, and falls back to zero as certainty returns. That shape is what bounds the probing band on both sides.

3.5 The bands that come out

Sweeping p from 0 to 1 in twenty thousand steps and recording every point where the chosen action changes gives the bands the system actually uses. Nobody selected these

numbers. They came out of the frozen cost table and the calibrated bite rates, and they move on their own if either input changes.

$$PASS \ p < 0.0647 \quad BAIT \ 0.0647 \leq p < 0.8793 \quad DIVERT \ p \geq 0.8793$$

The middle action exists only under EVSI; cost accounting alone gives one boundary

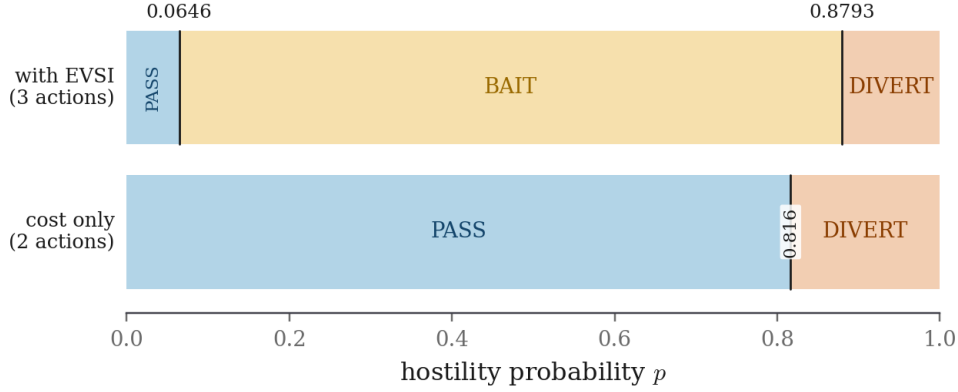


Figure 3. The derived bands, above, compared with the same axis under cost accounting alone, below. Removing the information term leaves a single cut-off at 0.816 and no middle band anywhere. Notice also that the divert edge moves upward rather than downward.

That last observation is the safety argument in one sentence. Because the probing band sits underneath the divert threshold and pushes it up from 0.816 to 0.879, a system that probes is more reluctant to divert an honest user than the same system without a probe, not less. Adding deception made the system safer for ordinary visitors, and that is a consequence of the arithmetic rather than a design goal we aimed at.

4. RESEARCH OBJECTIVES

The objectives below are written so that each one can fail. That matters more than it sounds: an objective that cannot fail is not an objective, it is a description. Table 8 names the measurement that would settle each one, and the section where that measurement appears.

7. **RO1 — Establish whether the third action can be derived rather than tuned.**
Work out the belief interval over which probing is the cheapest action, given only a frozen cost table and a measured probe effectiveness. Then check whether that interval exists at all when the information term is removed. If it does exist under cost accounting alone, the central claim of the project fails.
8. **RO2 — Test whether the conclusions survive the one estimated parameter.**
One number in the model, the attacker base rate, has to be estimated rather than measured. Sweep it across its full plausible range and check whether the probing band stays non-empty and the divert threshold stays at or above the cost-only boundary throughout.
9. **RO3 — Measure what the mechanism costs honest users.** Report the benign diversion rate, the benign probe exposure rate and the benign bite rate with confidence intervals, broken down by persona rather than only in aggregate, against a benign population deliberately built to look guilty.
10. **RO4 — Establish the effect of the probe causally, not by comparison.**
Randomly withhold the probe from a fraction of the sessions that reach the probing band, and estimate the difference between the two groups at the same belief state. This is what allows us to say the probe caused the effect, rather than that a system containing a probe scored higher.
11. **RO5 — Build a decoy that does not contradict itself, and measure it.** Interrogate the decoy repeatedly through different routes, count the disagreements, and report the rate. Then show whether the guarantee depends on which generator produces the fake content, by swapping it for a completely different one.
12. **RO6 — Characterise the limits honestly.** Report where the gain concentrates, where probing costs detection instead of adding it, and what happens against an adversary who already knows the probe exists.

Table 8. *Objectives, the evidence that settles each one, and where it is reported.*

#	Objective	Measurable outcome	Reported in
RO1	A derived third action	Band edges computed from the cost table and calibrated bite rates; empty band under cost accounting alone	§3.3, §3.5
RO2	Robustness to the estimated	Band stays non-empty and divert floor stays at or	§6.5

#	Objective	Measurable outcome	Reported in
	parameter	above 0.816 across the full range of the attacker base rate	
RO3	Cost to honest users	Benign diversion rate with a confidence interval, broken down by persona; probe overhead in milliseconds	§5.6, §6.6
RO4	Causal effect of the probe	Randomised holdout; difference in divert rate at the same belief band; Fisher exact test	§6.4, §6.6
RO5	Decoy self-consistency	Contradiction rate over an adversarial fuzzer sweep, with and without the fact store, under two different generators	§5.8, §6.6
RO6	Honest characterisation of limits	Discordant-pair analysis by attack subcategory; adaptive-adversary sweep; the cookie-refusal evasion	§6.6, §7

5. METHODOLOGIES

This section walks through the system one step at a time, in the order a request meets each part. Seven components in total, and only the fourth one contains anything learned from data.

5.1 Step 1: working out which session a request belongs to

Suspicion builds up per session, not per request, so the first job is deciding which requests belong together. The proxy sets a cookie on first contact and keys the session on that cookie, with a thirty-minute idle timeout.

A fallback exists that would group requests by a rough fingerprint made from the user-agent string, the address and a few headers, but it is switched off by default. The reason is practical. On a single test machine every traffic generator runs on loopback and shares the same fingerprint, so turning it on would merge dozens of separate clients into one session and corrupt every per-session measurement we make.

That default has a real cost, and we state it rather than hide it. A tool that refuses cookies gets a fresh session on every single request, so its score is reset each time and can never build up. We measured exactly this happening with a real attack tool, and we report it in Section 6.6 along with the one-flag mitigation.

5.2 Step 2: turning a request into numbers

Each request becomes eighteen numbers. They are cumulative over the session rather than computed fresh each time, and that is what lets a series of individually unremarkable requests add up to a confident conclusion without any hand-tuned decay constant anywhere in the system. The features are split into two groups on purpose, because the two scores need different evidence.

Table 9. *The eighteen features, grouped by the score each one feeds.*

Group	Features	What they capture
Automation, 10 features	auto_interarrival_last, auto_interarrival_cv, auto_requests_per_min, auto_asset_fetch_ratio, auto_fetched_assets, auto_browser_header_ratio, auto_header_count, auto_ua_is_tool, auto_ua_stable, auto_cookie_carried	Rhythm and browser-like behaviour. People are irregular, scripts are metronomic. The asset-fetch ratio is the single strongest signal, because a browser pulls stylesheets and images while a script does not bother
Malice, 8 features	mal_input_length, mal_db_keywords, mal_special_chars, mal_encoded_depth, mal_error_ratio, mal_distinct_paths, mal_param_tamper, mal_distinct_usernames	The shape of what is being sent: how long the input is, how many database keywords and special characters it contains, how deeply it is encoded, what

Group	Features	What they capture
		proportion of replies were errors, and how widely the session ranges across pages and accounts

Two changes to this list were forced by measurement rather than by taste, and both are recorded here rather than folded quietly into a version number. They are, in our view, the most honest part of the project.

Version 3 removed two features. One counted how long a run of ascending record identifiers the session had walked, and the other simply flagged any access to an API path. Together they diverted 100% of benign JSON-API integration clients. The reason is straightforward once you see it: a legitimate reporting integration walks record identifiers in ascending order over the API, which is character for character the same shape as an attacker sweeping through other people's records. No passive feature separates those two honestly, so we removed both features and handed that detection to the probe instead. That is exactly what a probe is for.

Version 4 then added one feature: a count of how many distinct usernames the session has tried. Earlier drafts of this work reported that a forgetful user was wrongly diverted three times in five, and argued that this was the unavoidable price of having a hard negative in the corpus. That argument was wrong. A forgetful user fails against one account and then succeeds, while an attacker spraying stolen passwords walks through many accounts, and no earlier feature looked at that axis at all. Adding it removed the last remaining benign false positive and lifted passive recall from 0.80 to 0.915 at the same time. The price of that fix is stated in Section 7.

5.3 Step 3: the dual suspicion meter

Two logistic models run over the two feature groups and produce two scores, both between 0 and 1 and both accumulating across the session. One says how automated the visitor looks, the other how hostile.

Keeping them apart is not cosmetic, and the justification is easy to state. A vulnerability scanner is highly automated and highly hostile. A price-comparison bot is highly automated and completely harmless. A careful human attacker is barely automated and extremely hostile. One combined score cannot express those three situations, because the first and second would look identical to it. Two scores can. In our system the malice axis

supplies the belief that drives the decision, while the automation axis decides which probe to choose.

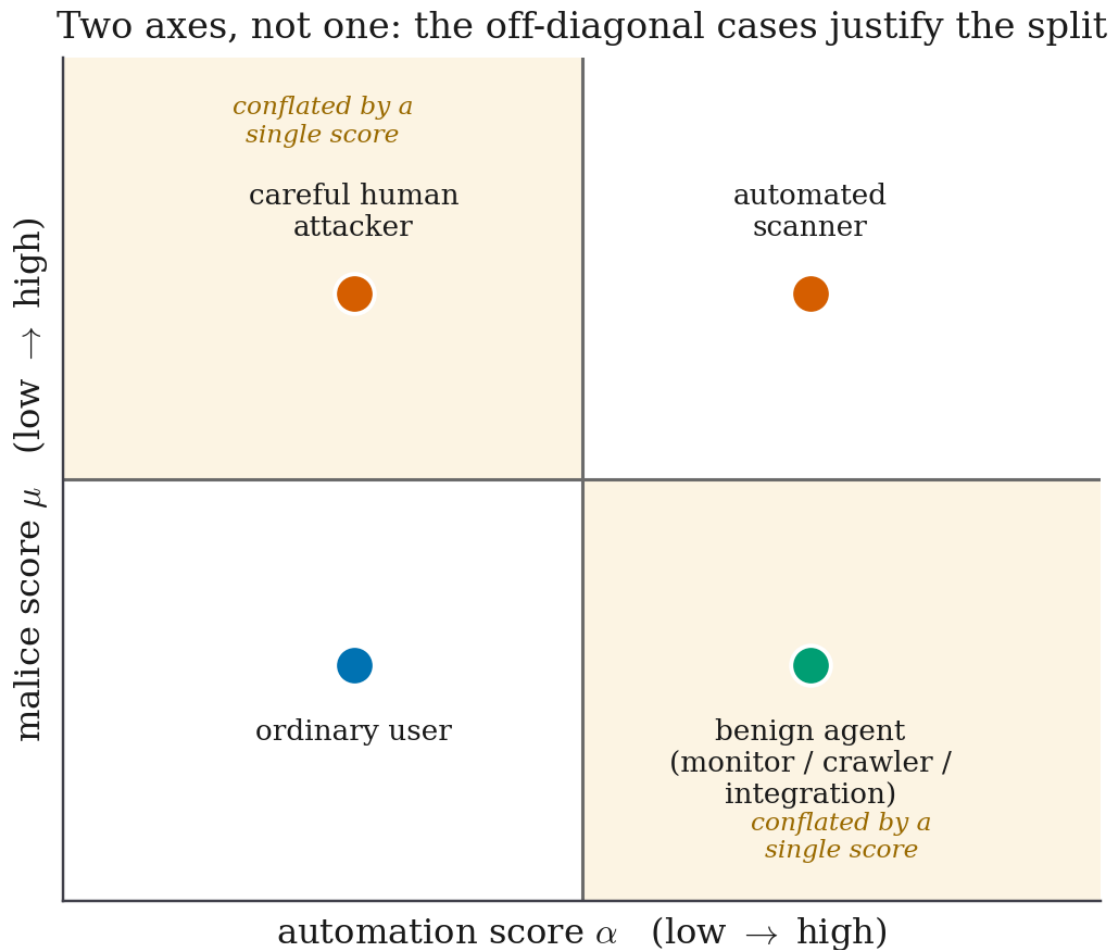
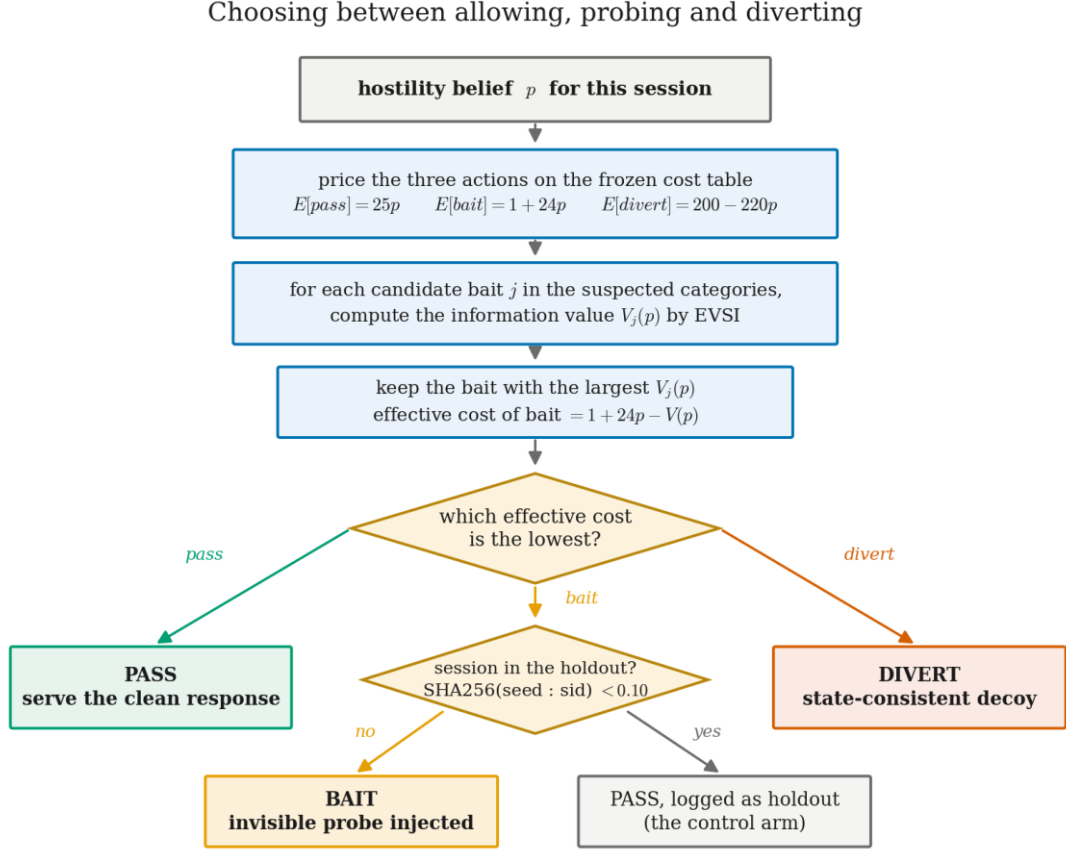


Figure 4. *The two-axis suspicion model. The corpus contains all four quadrants deliberately. If it contained only hostile scripts and hostile humans, then automation and malice would be perfectly correlated, a single score would work just as well, and the second axis would be impossible to defend.*

5.4 Step 4: the decision policy

The policy prices the three actions against the frozen table, works out the information value of each candidate probe, keeps the best one, and picks whichever effective cost is lowest. Figure 5 shows the complete path, including the holdout branch that Section 6.4 explains.



Only BAIT has its information value subtracted, because PASS and DIVERT end the session's decision.

Figure 5. The decision rule as it is actually implemented. Candidate probes are scored by their information value and the best one is kept. Only the probing action has that value subtracted from its cost, because passing and diverting end the decision.

5.5 Step 5: the bait library and its calibration

Six probes were built across three attack categories, each riding on a different part of the reply: fabricated SQL error text, an unused JSON field, an HTML comment, and a hint at an endpoint that does not exist. Each one carries a unique token, so a bite can be traced back to the exact probe and the exact session it came from. Five of the six survive in the current library. The sixth was withdrawn during recalibration, for the honest reason that this particular target exposes no JSON authentication surface for it to ride on, so it could never have fired here.

How well each probe works is measured, not assumed. A dedicated calibration round, kept separate from both the training round and the final test round, estimates how often attackers bite and how often benign visitors bite, per category, and freezes the result. The policy refuses to produce reportable numbers from uncalibrated guesses, and that refusal

is enforced in code rather than left to the researcher’s discipline. Table 10 gives the frozen values.

Table 10. *Calibrated probe effectiveness. The likelihood ratio for a bite is how many times more likely a bite is from an attacker than from a benign visitor, and it is exactly the weight added to the suspicion score when a bite happens.*

Probe	Rides on	Bite rate, attacker	Bite rate, benign	Ratio for a bite	Ratio for no bite
B-SQL-1	Fabricated database error text	0.563	0.0081	69.5	0.440
B-SQL-2	Fabricated database error text	0.512	0.0038	134.9	0.489
B-IDOR-1	Unused JSON field	0.794	0.0192	41.4	0.210
B-IDOR-2	HTML comment naming a record view	0.753	0.0067	112.3	0.249
B-AUTH-1	Hint at a deprecated login endpoint	0.580	0.0056	103.5	0.423

Two things in that table are worth reading carefully. The benign bite rates are all well under 2%, which is what makes probing cheap enough to be worth doing at all. And the ratios for a bite are large, between 41 and 135, which is why a single bite can move the suspicion score far enough to cross the divert edge in one step, as the worked example in Section 1.2.4 showed.

Reporting the ratio for declining a probe matters as much as reporting it for a bite, and this is a point most of the literature skips. A system that only ever revises suspicion upward will accumulate without limit and will eventually divert somebody for browsing slowly. Declining a probe is weak evidence of innocence, and our model says so: the values in the last column run between 0.21 and 0.49, so a visitor who ignores a probe has their score nudged down rather than left unchanged.

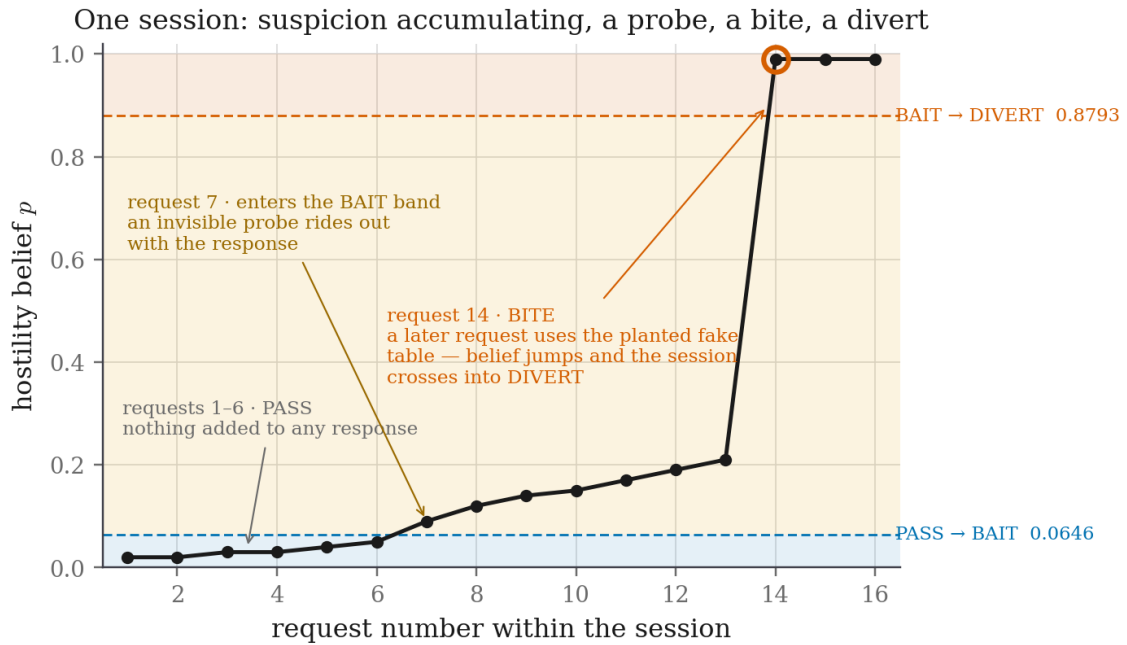
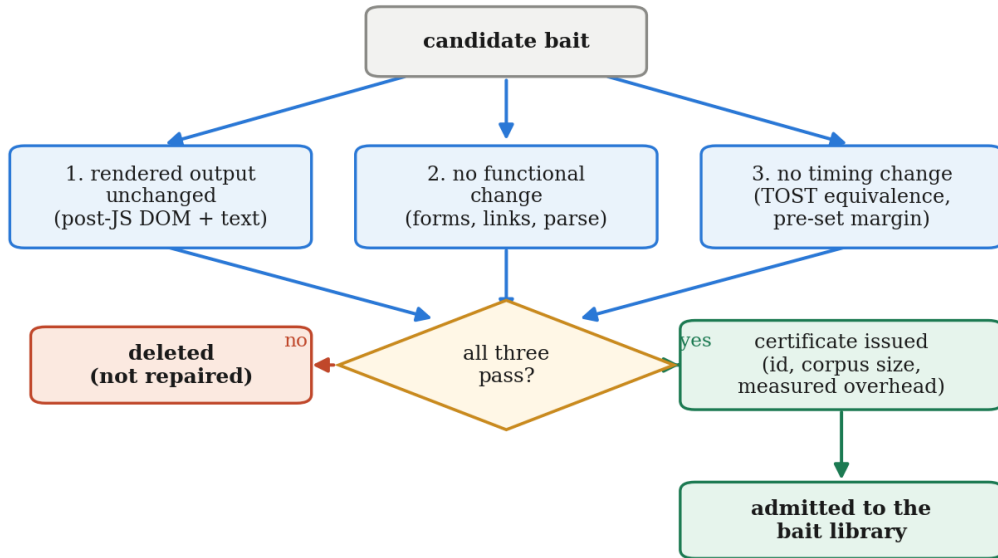


Figure 6. One session from beginning to end. Suspicion accumulates over the first few requests, the session enters the probing band and a probe rides out with an ordinary reply, and a later request uses the planted fake table name. The belief jumps and the session crosses into diversion.

5.6 Step 6: the invisibility gate

This is the most important safety mechanism in the project, and it was deliberately built before any probe existed. The ordering matters. A probe that leaks into the normal user experience does not merely weaken our results, it destroys the central claim, which is that provoking is safe. So the rule is strict: a probe that fails the gate is deleted, not repaired.

The invisibility gate — three tests, verified before use



The gate is built and certified BEFORE any bait exists (spec §13.1); the engine re-checks the certificate at run time.

Figure 7. The three tests a candidate probe must pass before it is allowed to exist at all. The timing ceiling of half a millisecond sits far below the natural variation of a real reply, so a baited reply cannot be told apart from a clean one by timing it.

The third test deserves a note, because it is a statistical point that is easy to get wrong. “We found no significant difference” is not evidence of invisibility. It is absence of evidence, and with a small enough sample you can fail to find a difference that is really there. The correct tool is an equivalence test. We use two one-sided tests [38] against a bound fixed in advance, which lets the gate state and defend a bound rather than merely fail to reject one.

Table 11. Invisibility certificates. Every probe is certified against 116 real benign replies.

Probe	Passed	Median overhead (ms)	95th percentile (ms)	Margin below the 0.5 ms ceiling
B-SQL-1	Yes	0.0206	0.0301	24×
B-SQL-2	Yes	0.0114	0.0123	44×
B-IDOR-1	Yes	0.1655	0.1867	3.0×
B-IDOR-2	Yes	0.0121	0.0160	41×
B-AUTH-1	Yes	0.0115	0.0145	43×

The withdrawn sixth probe also held a passing certificate at 0.1947 ms median overhead. It was removed from the library for lack of a surface to ride on rather than for any failure of the gate, and its certificate remains on file.

Two guards make the rule real while the system is running, rather than only at build time. The engine refuses to serve any probe that does not have a passing certificate on file, and a probe that cannot be injected cleanly into a particular reply degrades to the clean reply rather than to a broken one. An earlier version of the gate certified against only thirteen replies, which is far too small a sample to support a safety claim. We closed that gap, and the corpus size is now recorded inside every certificate next to the overhead numbers so that an examiner can check it.

5.7 Step 7: spotting a bite and updating the belief

Every incoming request is checked against every token planted so far. First the requesting session's own probes, then every other session's. A token that turns up outside the session it was issued to means either that a probe leaked somewhere public or that an attacker is rotating identities, and both of those are more informative than an ordinary bite, so both are flagged separately.

The update itself is a likelihood ratio applied in log-odds, which is simply the space where multiplying by a ratio becomes adding a number. The original specification said the malice score should “jump sharply” on a bite but never said by how much, and that missing number was a real gap. It is now the calibrated ratio belonging to the specific probe that was bitten. With no bites at all, the score is exactly what the meter produced on its own.

One ordering detail matters. Bite detection runs before scoring, so a request that acts on a probe raises suspicion on that same request rather than on the next one. Without that ordering the attacker would get one free request after every bite.

5.8 Step 8: the decoy and its fact store

Once a session is diverted it stays in the decoy for every later request. It is never blocked and never shown an error, so the attacker has no reason to suspect they were caught.

The hard part is not generating a plausible fake world. It is remembering it. Fake environments that generate replies on the fly forget what they said, so asking the same question twice, or asking it through a different page, returns two different answers. That inconsistency is exactly how an experienced attacker realises they are inside a trap.

Our fact store maps a namespace and a key to a value. When a fact is needed for the first time it is produced by a generator seeded from the run seed, the namespace and the key together, so the same key always produces the same value even if it is first reached months later through a completely different endpoint. The value is then written down, first writer wins, so two simultaneous readers always agree. If the attacker writes something themselves, their write is stored at a higher precedence, so a generated default can never overwrite a change the attacker made and then read back.

Table 12. *The four kinds of consistency a decoy needs, and how each one is guaranteed.*

Dimension	What must hold	How it is guaranteed
Repetition	Asking the same thing twice returns the same answer	A deterministic generator plus first-writer-wins storage
Cross-reference	A person named on one page exists with the same details on every other page	The same key produces the same value across all endpoints
Write-then-read	If the attacker changes something, reading it back returns what they wrote	Attacker writes are stored at a higher precedence than generated values
Referential integrity	An identifier mentioned in one record resolves to a matching record elsewhere	Generators resolve references through the fact store rather than inventing them

Nothing is ever generated inside the request path. A model call in the middle of a reply would make the decoy slower than the real site, and a slow decoy is a detectable decoy. The generator behind the fact store is deliberately pluggable, and Section 6.6 reports what happened when we swapped the deterministic one for a stochastic language model.

5.9 Step 9: tamper-evident logging

Every decision is written to an append-only store in which each record contains a hash of the record before it [36]. Changing or deleting an earlier entry therefore breaks the chain at a detectable point. Each record holds the session identifier, the full request, the eighteen extracted features, both scores before and after the update, the action chosen, the reason it was chosen, which probe was injected if any, whether a bite occurred, and the reply that was served.

A verifier walks the chain and reports the first index where it breaks. This is what makes the evaluation auditable rather than merely reported, and it is also what let us reconstruct the false positives described in Section 5.2 after the fact instead of having to re-run everything.

6. EXPERIMENTAL SETUP

6.1 The target application and the stack

The target is a deliberately weak staff-directory website. It has ordinary server-rendered pages plus a JSON API, a login flow with a predictable second factor, a search box built by joining strings together into a database query, and profile pages keyed by a simple sequential number. It knows nothing at all about the defence, which keeps the two independent.

The proxy sits in front of it on a different port, and clients only ever talk to the proxy. The decoy is a full copy of the target backed by the fact store, and the consistency fuzzer confirmed there are no structural differences between them that an attacker could use to tell which one they are talking to.

6.2 Building the corpus

No public dataset can support the numbers this project reports, so a purpose-built corpus was unavoidable. As Section 2.1.2 explained, CSIC 2010 has no sessions, no timing, no benign but automated traffic and no bait interactions, and every one of those is needed here.

The corpus is produced by a single command that wipes previous state, seeds a fresh database, starts a private server, runs every generator, stops the server, assembles the corpus, and then verifies that every traffic record actually joined to a label. That last check exists because a silent labelling failure is the kind of mistake that is only discovered after all the results have been written up.

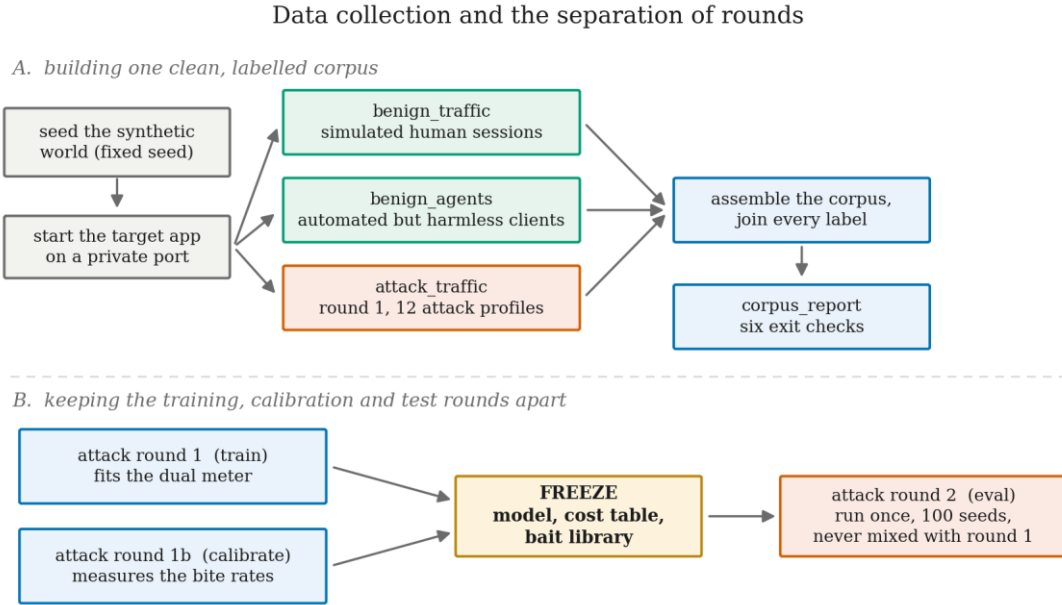


Figure 8. *Corpus construction, and the separation of the three attack rounds. The round-1 generator physically refuses to produce evaluation data, so a rerun of the training corpus cannot quietly become the test set.*

The corpus is not accepted until it passes six automated checks: at least fifty benign sessions; some benign but automated traffic present; asset fetching actually separating humans from scripts; human think-times present and plausible; humans measurably more irregular than scripts; and every single record labelled. These are the exit conditions for the phase, and they are the evidence an examiner will want to see.

6.3 A benign corpus built to be hard

The benign diversion rate carries the safety half of this entire project, and it is trivially zero if the benign population only browses gently. An examiner will ask exactly that, and they should. A near-zero false-positive rate is only interesting in proportion to how hard the negatives were, so we built the benign corpus to be awkward on purpose.

Table 13. *Benign personas in the corpus, and the attack each one resembles.*

Persona	Share	What it does	Which attack it looks like
normal	78%	Ordinary browsing with human pauses between clicks	—
apostrophe_searcher	12%	Looks up a colleague named O’Connell	An injection probe. We verified that it returns the identical verbose database error and HTTP 500 that a real attacker sees
forgetful	10%	Fails to log in three to five times, then succeeds	A credential attack, and the exact trigger condition for one of our auth probes
ReportingIntegration	agent	Walks record identifiers in	A record-sweeping attack, differing only in that every identifier genuinely belongs

Persona	Share	What it does	Which attack it looks like
		ascending order over the API	to it
UptimeMonitor	agent	Polls at a fixed interval, no cookies, no images	A scanner. Every automation feature fires at once
SearchCrawler	agent	Automated search traffic	Content scraping

The first of those is the one worth putting in a paper. A staff member looking up a colleague whose name contains an apostrophe produces a reply that is identical in kind to what an attacker sees while probing for injection. Any system that can separate them has learned something about intent and context rather than about the shape of a payload. And if it cannot separate them, that is a genuine finding about the limits of response-level detection, which should be reported rather than quietly excluded from the corpus.

These persona names are recorded in a notes field and never in the class labels. An awkward honest user is exactly as benign as a straightforward one. Encoding “this one looked suspicious” into a label would teach the meter that unusual means hostile, which is precisely the brittle heuristic this whole project exists to replace. Keeping the persona in a note still lets us report where the false positives concentrate, which is far more useful than a single aggregate number.

6.4 Arms, rounds and the randomised holdout

Baselines are selected by one flag in the configuration file, and every arm runs through the same code path. That is what makes the comparison honest: there is no separate implementation that might differ in some unnoticed way. As a second layer of protection, in any mode other than the full system the probe engine is never even constructed, so a baseline is provably probe-free rather than relying on the policy arithmetic to keep it that way.

Table 14. *The five evaluation arms.*

Arm	What it is	Scoring	Probe	Decoy
B0	The proxy forwards everything. No defence at all	No	No	No
B1	A signature web application firewall: regular expressions for injection, scripting, traversal and command injection, plus scanner user-agents	Rules	No	No
B2	Full scoring, no probe, no decoy. This is the honest baseline	Yes	No	No
B3	Passive scoring with a static decoy, which is roughly the published state of the art	Yes	No	Static
B4	The full system. This is the contribution	Yes	Yes	Consistent

Comparing B4 with B2 answers a useful question, but not a sharp one. The two arms differ in every component, so any difference between them is tangled up with all of those differences at once. It is a comparison between two systems presented as evidence about one mechanism.

The randomised holdout answers the sharper question. A fixed fraction of the sessions that reach the probing band are deliberately not probed, and they are logged as holdout so that they can be told apart from sessions that were not probed because the policy chose PASS. Assignment is a hash of the run seed and the session identifier, which gives us two things at once: it is fully deterministic, so a run can be replayed exactly, and it is unpredictable to anyone who cannot see the seed, so an attacker cannot arrange to land in the control group.

Because assignment is random among sessions that have all reached the same belief state, the difference between the two groups is an unbiased estimate of the effect of probing itself. This costs us some detection performance by design, since a fraction of real attackers deliberately go unprobed, and we report that cost rather than absorbing it quietly. In exchange, we can say that the probe caused the effect, which is something almost nothing else in this literature can say.

6.5 Metrics and statistical protocol

Expected cost per session is the primary metric rather than accuracy, for the reason Axelsson [32] set out: attacks are rare, so even a very accurate detector produces mostly false alarms, and weighting by cost is the standard answer to that. Precision, recall and F1 are reported as well so that the work can be compared with the existing literature, but they do not lead.

Table 15. *Metrics used, with the target for each.*

Metric	Definition	Target
Expected cost per session	Average of $C(\text{action}, \text{truth})$ over every session	Lower than the passive baseline
Benign diversion rate	Benign sessions diverted, divided by all benign sessions	Effectively zero, always with an interval
Benign probe exposure	Benign sessions shown a probe, divided by all benign sessions	Low, and reported either way
Benign bite rate	Smoothed as $(k + \frac{1}{2}) / (n + 1)$ so a zero is never reported as certainty	Zero, never assumed
Probe overhead	Median extra delay added per probe	At most 0.5 ms
Requests to decision	The first request number at which the action becomes DIVERT	Lower when the session is probed
Attack recall	Attack sessions diverted, divided by all attack	High

Metric	Definition	Target
	sessions	
Contradiction rate	Fuzzer probes that disagreed, divided by all fuzzer probes	Below 1%

The statistical protocol was fixed before the runs, not chosen afterwards to suit the numbers. Every arm is evaluated over 100 independent seeded traffic draws against the one frozen model, with 120 attack and 80 benign sessions per draw. Every arm sees byte-identical traffic on every seed, and that matching is what licenses a paired test rather than an unpaired one.

Proportions carry Wilson intervals, which behave sensibly near zero where an ordinary interval does not. The comparison between the passive baseline and the full system uses an exact paired McNemar test, because each attack session forms one matched pair and only the pairs where the two arms disagreed carry any information. The holdout uses a Fisher exact test with a bootstrap interval on the difference. The primary comparison is named in advance and everything else is labelled exploratory, so that we are not quietly running many tests and reporting the best one.

One result needs no sample size at all, and it is reported first for that reason. The probing band and the divert floor are consequences of the cost table and the calibrated bite rates, so they hold no matter how many sessions were run. The single number that had to be estimated rather than measured is the attacker base rate, and Figure 9 sweeps it across its whole plausible range.

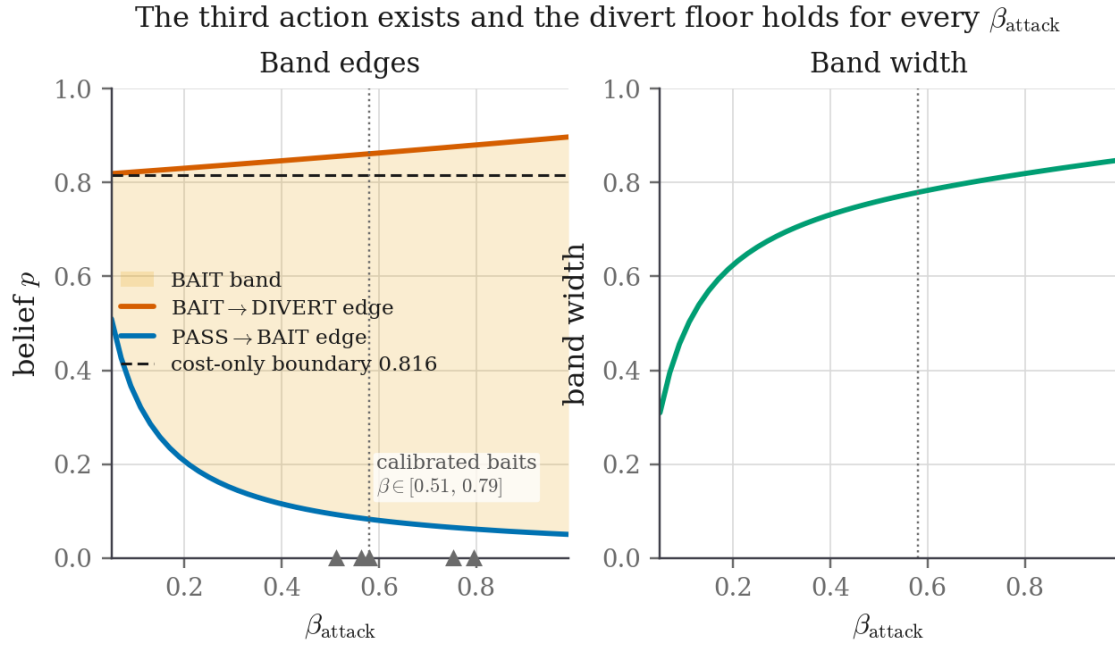


Figure 9. How the derived bands respond to the attacker base rate. Across the entire range the probing band stays non-empty and the divert edge stays above the cost-only boundary of 0.816. Only the width of the band and the strength of the likelihood ratio move. Neither the existence of the third action nor the safety margin it provides depends on the estimate.

6.6 Results obtained so far

All eight implementation phases are complete and the evaluation has been run. The numbers below are pooled over 100 seeds per arm on identical traffic draws, which comes to 12,000 attack sessions and 8,000 benign sessions in each arm.

Table 16. Headline results, each with the test that supports it. These are from the completed run against the previous bait library; Section 6.6.5 explains why they are being regenerated and what it does and does not affect.

Result	Value	Test	Significance
Causal effect of the probe** (randomised holdout)	+0.029, 95% CI [+0.012, +0.046]	Fisher exact	p = 0.00024
Attack recall, full system against passive	0.933 [0.929, 0.938] against 0.915 [0.910, 0.920]	Paired McNemar, 499 against 280 over 12,000 pairs	p < 10⁻⁴
Consistency across seeds	Full system ahead in 78 of 100 paired seeds	—	—
Where the probe acts	The gain is confined to UI record-access sessions (0.704 to 0.807)	Paired McNemar	p < 10⁻⁴
Benign diversion	Passive 4 of 8,000; **full system 0 of 8,000	Wilson interval	The probe adds zero false positives
Band non-empty and divert floor holds	Invariant across the whole base-rate range	Parameter sweep	Structural
Decoy contradiction rate	0% with the fact store, 100% without it	Fuzzer, 286 probes	—
The same with a language model	0% with, 100% without	15 entities, local model	Generator-independent

6.6.1 The causal estimate

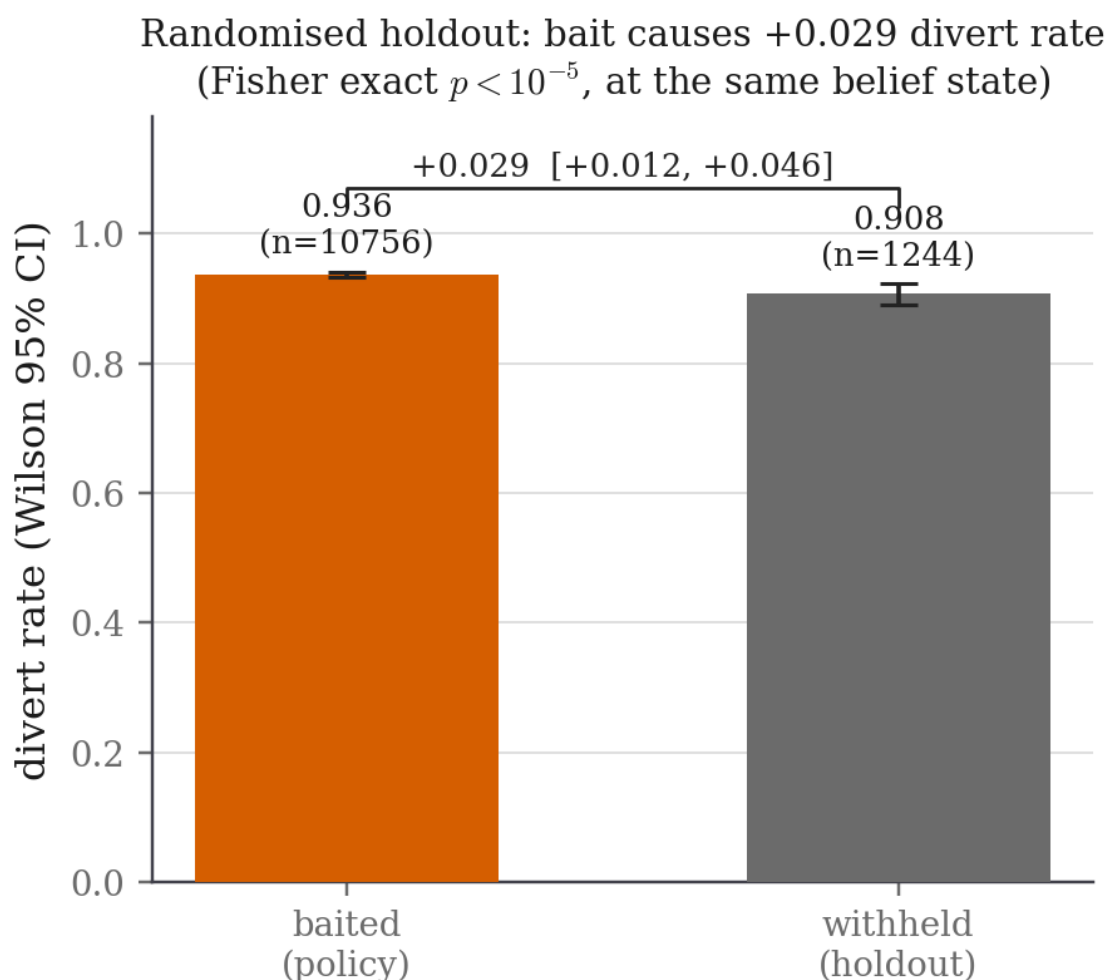


Figure 10. *The causal estimate. Probed and withheld sessions sit at the same belief band, so the gap between them is attributable to the probe rather than to any other difference between two systems.*

The effect is smaller than earlier drafts of this work reported, and the history is worth stating plainly. A single traffic draw once gave +0.23, a twenty-seed run gave +0.11, and the previous bait library gave +0.053. Two things shrank it. The passive meter is strong enough on its own that, with the baseline already catching 91.5% of attacks, there is little headroom left for a probe. And the recalibration described in Section 6.6.5 raised the divert threshold, so the probe now defers more diversions than it used to. What survives all of that is +0.029, and it is still significant. We report it because it is the honest figure, not because it is the flattering one.

6.6.2 Detection, and exactly where the gain lives

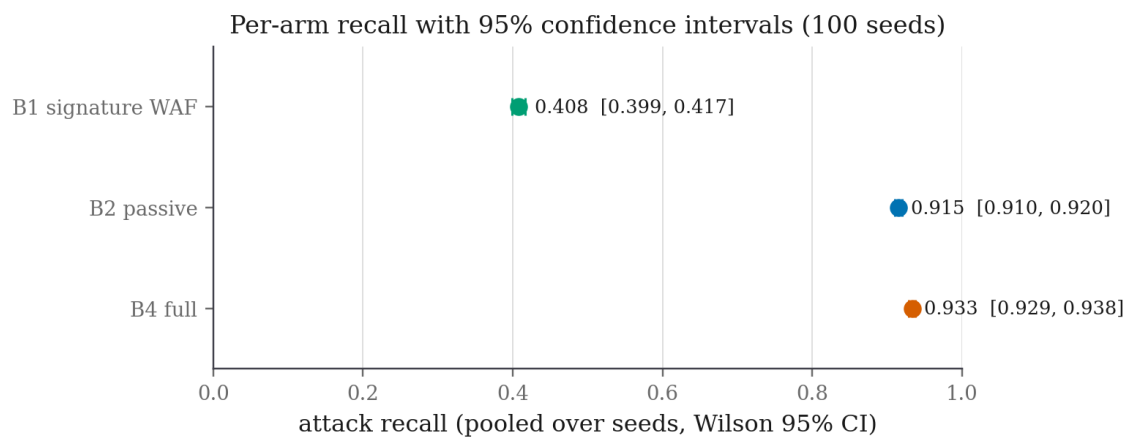


Figure 11. Attack recall for each arm with 95% confidence intervals. The signature firewall reaches only 0.408, because round-2 attacks are obfuscated and, decisively, because record-access attacks are perfectly valid syntax and leave it nothing to match on. The passive and full intervals do not overlap.

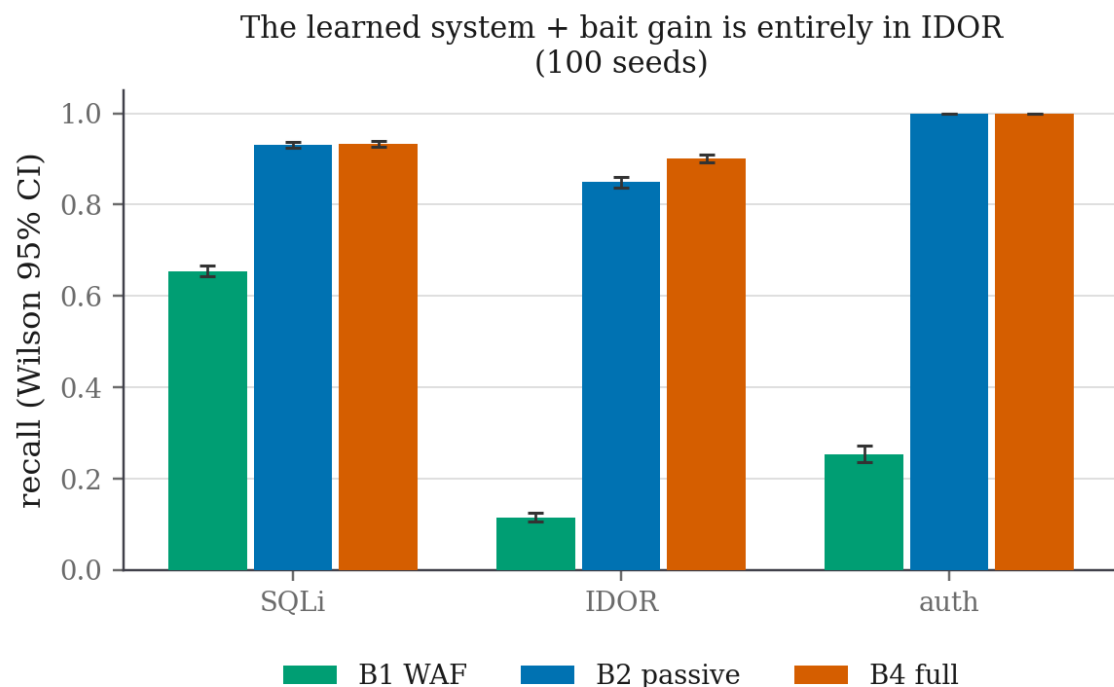


Figure 12. Recall broken down by attack category. The learned system lifts every category well above the firewall, but the probe's gain over passive scoring lives almost entirely in record-access attacks, rising from 0.850 to 0.902.

Splitting the matched pairs by subcategory sharpens that considerably. The whole net gain sits in UI-based scattered record access, which goes from 0.704 to 0.807, and that is precisely the case the derivation predicts: the one category a signature cannot see, and the one where the passive meter is genuinely uncertain. Everywhere else the probe is neutral. The low-and-slow injection prober bites 22% of the time and its diversion rate does not

move at all, because those sessions were already caught, and scattered access over the API is caught passively because an attacker walking identifiers hits many missing records while a benign integration reads only the ones that exist.

One subcategory cannot reach the probe at all, and it deserves to be stated rather than buried. Obfuscated injection has a bite rate of exactly 0.000 even though nine in ten of its sessions are shown a probe, and it accounts for 405 of the full system's 799 remaining misses. The cause is in our attacker simulation, not in the defence: those profiles fired payloads and never read the response body, so a response-side probe could not reach them by construction. *An attacker that injects and never reads the error is not performing error-based injection at all.* That has been corrected, and the run measuring it is deliberately separate from the numbers reported here.

6.6.3 Safety

The safety half held. The full system diverted no benign session at all out of 8,000, and precision was 1.000 for every learned arm. Ninety per cent of benign sessions were shown a probe and not one of them acted on it, which is the invisibility gate holding at run time and at scale rather than only on the bench.

Exposure being high is not a problem, and it is worth being clear why. Exposure is harmless precisely because the probe is invisible. The bite rate is the number that matters, and it was zero across every benign persona, including the two hard ones.

6.6.4 Two further checks

We ran the frozen system against real off-the-shelf attack tools that we did not write and cannot bias. sqlmap was diverted on its third request and wapiti on its thirteenth, and neither of them bit a probe. That is exactly what our model predicts and what our scope claim requires: automated tooling is caught by the passive meter, and the probe exists for the human and semi-automated adversary that a scanner is not.

One tool, ghauri, refused the proxy cookie, so every one of its 1,321 requests arrived as a fresh session and its score was reset each time. This is the evasion Section 5.1 anticipates. Even so, its payloads were damning enough that 46% of its individual requests were diverted on the spot. Turning on the fingerprint fallback collapsed it to a single fully diverted session. The evasion is real, it was anticipated in advance, and the mitigation is a configuration flag rather than a redesign.

Separately, we retested the fact store’s guarantee with a stochastic language model in place of the deterministic generator, running locally on loopback with nothing in the request path. The contradiction rate was 0% with the fact store and 100% without it, in both cases. Consistency is therefore a property of the store rather than of the generator behind it, which is the point of the design.

6.6.5 An honest note on the current state of the numbers

While preparing this synopsis we found a defect in the calibration harness. It is worth being clear that the defect is in the measuring apparatus, not in the system being measured.

The simulated bait-following attacker used during calibration bit whichever planted token it encountered first, rather than the one belonging to the category being measured. Because every session warms up through the login flow, and those replies carry the authentication and record-access probes, an authentication follower was sometimes diverted during warm-up before it ever reached its own probes. That put the wrong sessions in the denominator and understated one probe’s effectiveness by roughly a factor of seven.

Recalibration is now complete and the library has been re-frozen with five probes. The corrected bite rates are the ones already given in Table 10, and they shift the derived band from [0.0516, 0.8626] to [0.0647, 0.8793]. Every figure in this synopsis is drawn from that re-frozen library, so the structural results reported here are the current ones.

The full evaluation has since been re-run against the re-frozen library, and every measured number in this synopsis comes from that re-run. The effect of the correction was to make the results *worse*, and that is the correction doing its job. Attack recall for the full system fell from 0.946 to 0.933, the causal effect fell from +0.053 to +0.029, and the count of sessions where the probe defers a diversion the passive rule would have made rose from 72 to 280. The earlier figures were inflated by the measurement bug described above.

The reason for the drop is mechanical rather than mysterious. A higher divert threshold means the system waits longer before sending anyone to the decoy, so a few attackers who would have been diverted immediately are instead probed first and, within a finite

evaluation horizon, not caught. That is the same conservatism that keeps the benign diversion rate at zero, and we would rather report it than trade it away.

It is worth being clear about which claims this touches. The derived band, its invariance to the attacker base rate, and its invariance to the cost ratio are properties of the decision rule and of the frozen inputs, so the correction does not affect them at all. Only the measured recall and holdout figures moved. There is also independent support for the direction of the correction: a separate targeted experiment measured a response-reading brute-force attacker at 0.95 diverted against the old model, which is consistent with the corrected bite rate and not with the old one.

6.7 Implementation status

Table 17. *The eight phases, each with the condition it had to meet before the next one began.*

Phase	Component	State
0	Cost table, label schema, logging skeleton	Complete. Frozen and enforced by a hash check at start-up
1	Target application and benign traffic generator	Complete. All six corpus exit checks pass
2	Attack round 1, the training corpus	Complete. Twelve profiles, all four automation-by-malice cells populated
3	Features, dual meter, cost policy, proxy	Complete. Passive baseline validated end to end
4	Bait library and invisibility gate	Complete. The gate was built first, six probes certified, bite rates calibrated
5	Decoy, fact store and consistency fuzzer	Complete. Zero contradictions over 286 probes; full parity with the target
6	Integration, fail-open verification, model freeze	Complete. Per-component fail-open, model frozen behind a verified hash manifest
7	Attack round 2, baselines, ablations, results	Complete. Four arms over 100 paired seeds. Being re-run against the corrected library described in Section 6.6.5

7. CONCLUSION

This project asked whether a web-attack detector should have a third action, and whether the moment to take it can be worked out rather than guessed. The answer to both is yes, and the working out turned out to be more informative than we expected when we started.

Judged on cost alone, probing is never worth doing. It is strictly more expensive than passing at every level of suspicion short of certainty, and the rule collapses into a two-way choice with a single cut-off at 0.816. The probing band exists only because a probe buys information, and it closes again at both ends where the system is already sure, because an observation that cannot change your decision is worth nothing. That makes the trigger a consequence rather than a setting, which in turn means there is no threshold for an adversary to discover and stay underneath. Sweeping the one estimated number across its full range leaves the band non-empty and the divert floor above the cost-only boundary throughout, so neither the existence of the third action nor the safety margin it provides depends on a figure we had to guess.

The measurements support the derivation without overstating it. A randomised holdout gives a significant causal estimate of the probe's effect at a fixed belief state, which is something almost nothing else in this literature provides. Recall rises from 0.915 to 0.933, the confidence intervals do not overlap, the paired test is significant, and the gain sits in the one attack category no signature can see. The full system diverted none of 8,000 benign sessions while showing a probe to nine in ten of them, against a benign population built specifically to look guilty. The decoy does not contradict itself under adversarial interrogation, and that property survives replacing the generator underneath it.

7.1 Limitations

These are worth stating plainly. Naming a weakness yourself reads as rigour, while having it pointed out to you reads as an oversight.

- **The traffic is synthetic.** Both benign and attack traffic are generated rather than captured from a live website. We built the benign corpus to be as hard as we could make it and named the hardest negatives explicitly, but no generated corpus proves how a system behaves against real users. This is the main limitation of the work.
- **The scale is small.** One application, one machine, thousands of requests rather than millions. This is a controlled laboratory study, not an internet-scale deployment, and it is the reason a small logistic model is the right choice rather than a deep one.

- **No new decision theory is claimed.** The expected value of sample information is textbook, and we have said so at every point in this document. The contribution is its application to this decision.
- **The gain is not per-session.** The recall gain is significant in aggregate, but it is not dominance in every session. Two hundred and eighty of 12,000 matched sessions went the other way, because within the narrow band between 0.816 and 0.879 the probe defers a diversion that the passive rule would have made immediately. Those cases are reported next to the net effect rather than absorbed into it.
- **One attack category could not reach the probe at all.** Obfuscated injection accounts for half of all remaining misses and has a bite rate of exactly zero, because the attacker profiles simulating it never read the response body. A response-side probe therefore could not reach them however good it was. This is a limitation of our attacker simulation rather than of the mechanism, it has been corrected, and it is being measured separately rather than assumed away.
- **The robustness result is asymptotic.** Against an adversary who already knows the probe exists, the information value decays and the rule converges to the passive two-action rule. That is a statement about the limiting behaviour of the rule, not a guarantee about any individual session.
- **One evasion is real and measured.** A client that refuses cookies resets its own score under the default configuration. The mitigation is a configuration flag rather than a redesign, and both the evasion and the fix were measured.

7.2 What remains

Two things are outstanding before submission. The first is the re-run described in Section 6.6.5, against the corrected and re-frozen bait library. The second is the human deception study, for which a step-by-step protocol has already been written for a facilitator who does not know the code. Neither of them touches the structural spine of the work, because the derived band and its invariance are properties of the decision rule and hold regardless of how many sessions are eventually run.

8. TENTATIVE CHAPTER PLAN FOR THE PROPOSED WORK

Table 18 maps the proposed thesis onto the artefacts that already exist. The state column is deliberately honest about what is written, what is built but unwritten, and what is still moving.

Table 18. *Chapter plan, with the artefact backing each chapter and its current state.*

Ch.	Chapter	What it contains	State
1	Introduction	The two-action limitation and what it costs, the response-side probe, the four claims, and the negative result that motivates the whole approach	Drafted
2	Literature Review	The three deployed defence families and the three research lines, the placement table, the gap matrix, and the six gaps mapped onto the design	Complete
3	Threat Model and Scope	Attacker goals, knowledge and capability, the three attack categories, and what is deliberately left out of scope together with the reason	Drafted
4	System Design	The seven components, the two-axis meter, session identity, the fail-open ordering, and tamper-evident logging	Complete
5	The Decision Rule	The frozen cost table, the negative result under cost accounting alone, the information value, the derived bands, the properties and their tests, and both invariance sweeps	Complete
6	Calibrating the Probe	The bait library, the separate calibration round, the measured likelihood ratios in both directions, and the invisibility gate with its certificates	Complete, being re-frozen
7	The State-Consistent Decoy	The fact store, the four consistency dimensions, the fuzzer, the no-store ablation, and the language-model replication	Complete
8	Implementation and Reproducibility	The stack, the frozen artefacts and their hashes, determinism, and the test suite that enforces both	Complete
9	Evaluation	Corpus construction, the hard benign population, the five arms, the randomised holdout, the statistical protocol, and every measured result	Re-run in progress
10	Ablations	Removing the probe, removing the fact store, the same test under a stochastic generator, and the adaptive-adversary sweep	Three of four complete
11	Limitations and Threats to Validity	Split into limitations already eliminated and those that are irreducible, with the disagreeing-pair analysis and the cookie-refusal evasion	Drafted
12	Conclusion and Future Work	What was established, what is explicitly not claimed, and the human deception study as the next measurement	Outlined

9. REFERENCES

- [1] M. H. Almeshekah and E. H. Spafford, “Planning and Integrating Deception into Computer Security Defenses,” in Proc. 2014 New Security Paradigms Workshop (NSPW ’14), 2014, pp. 127–138. DOI: 10.1145/2683467.2683482
- [2] X. Han, N. Kheir, and D. Balzarotti, “Deception Techniques in Computer Security: A Research Perspective,” ACM Computing Surveys, vol. 51, no. 4, art. 80, pp. 1–36, 2018. DOI: 10.1145/3214305
- [3] J. Pawlick, E. Colbert, and Q. Zhu, “A Game-theoretic Taxonomy and Survey of Defensive Deception for Cybersecurity and Privacy,” ACM Computing Surveys, vol. 52, no. 4, art. 82, pp. 1–28, 2019. DOI: 10.1145/3337772
- [4] M. Zhu, A. H. Anwar, Z. Wan, J.-H. Cho, C. A. Kamhoua, and M. P. Singh, “A Survey of Defensive Deception: Approaches Using Game Theory and Machine Learning,” IEEE Communications Surveys & Tutorials, vol. 23, no. 4, pp. 2460–2493, 2021.
- [5] P. Beltrán López, M. Gil Pérez, and P. Nespoli, “Cyber Deception: Taxonomy, State of the Art, Frameworks, Trends, and Open Challenges,” IEEE Communications Surveys & Tutorials, 2025. DOI: 10.1109/COMST.2025.3594788
- [6] N. Provos, “A Virtual Honeypot Framework,” in Proc. 13th USENIX Security Symposium, 2004.
- [7] M. Nawrocki, M. Wählisch, T. C. Schmidt, C. Keil, and J. Schönfelder, “A Survey on Honeypot Software and Data Analysis,” arXiv:1608.06249, 2016.
- [8] A. Javadpour, F. Ja’fari, T. Taleb, M. Shojafar, and C. Benzaïd, “A Comprehensive Survey on Cyber Deception Techniques to Improve Honeypot Performance,” Computers & Security, vol. 140, art. 103792, 2024. DOI: 10.1016/j.cose.2024.103792
- [9] T. Barron and N. Nikiforakis, “Picky Attackers: Quantifying the Role of System Properties on Intruder Behavior,” in Proc. 33rd Annual Computer Security Applications Conference (ACSAC ’17), 2017. DOI: 10.1145/3134600.3134614
- [10] A. Vetterl and R. Clayton, “Bitter Harvest: Systematically Fingerprinting Low- and Medium-interaction Honeypots at Internet Scale,” in Proc. 12th USENIX Workshop on Offensive Technologies (WOOT ’18), 2018.
- [11] M. Sladić, V. Valeros, C. Catania, and S. Garcia, “LLM in the Shell: Generative Honeypots,” in 2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), 2024.
- [12] Reworr and D. Volkov, “LLM Agent Honeypot: Monitoring AI Hacking Agents in the Wild,” arXiv:2410.13919, 2024.
- [13] R. A. Bridges, T. R. Mitchell, M. Muñoz, and T. Henriksson, “SoK: Honeypots & LLMs, More Than the Sum of Their Parts?,” arXiv:2510.25939, 2025.
- [14] J. Yuill, M. Zappe, D. Denning, and F. Feer, “Honeyfiles: Deceptive Files for Intrusion Detection,” in Proc. 5th Annual IEEE SMC Information Assurance Workshop, 2004, pp. 116–122.
- [15] B. M. Bowen, S. Hershkop, A. D. Keromytis, and S. J. Stolfo, “Baiting Inside Attackers Using Decoy Documents,” in Proc. 5th Int. ICST Conf. on Security and Privacy in Communication Networks (SecureComm 2009), 2009, pp. 51–70. DOI: 10.1007/978-3-642-05284-2_4

- [16] A. Juels and R. L. Rivest, “Honeywords: Making Password-Cracking Detectable,” in Proc. 2013 ACM SIGSAC Conf. on Computer and Communications Security (CCS ’13), 2013, pp. 145–160.
- [17] S. Srinivasa, J. M. Pedersen, and E. Vasilomanolakis, “Towards Systematic Honeytoken Fingerprinting,” in Proc. 13th Int. Conf. on Security of Information and Networks (SIN 2020), 2020. DOI: 10.1145/3433174.3433599
- [18] R. Timmer, D. Liebowitz, S. Nepal, and S. Kanhere, “Evaluating Honeyfile Realism and Enticement Metrics,” ACM Transactions on Privacy and Security, vol. 28, no. 4, 2025. DOI: 10.1145/3763792
- [19] M. Kahlhofer and S. Rass, “Application Layer Cyber Deception Without Developer Interaction,” in 2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), 3rd Workshop on Active Defense and Deception, 2024.
- [20] M. Kahlhofer, S. Achleitner, S. Rass, and R. Mayrhofer, “Honeyquest: Rapidly Measuring the Enticingness of Cyber Deception Techniques with Code-based Questionnaires,” in Proc. 27th Int. Symp. on Research in Attacks, Intrusions and Defenses (RAID 2024), 2024. DOI: 10.1145/3678890.3678897
- [21] M. Kahlhofer, M. Golinelli, and S. Rass, “Koney: A Cyber Deception Orchestration Framework for Kubernetes,” in 2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), 2025, pp. 690–702.
- [22] C. Kruegel and G. Vigna, “Anomaly Detection of Web-based Attacks,” in Proc. 10th ACM Conf. on Computer and Communications Security (CCS ’03), 2003, pp. 251–261. DOI: 10.1145/948109.948144
- [23] W. Robertson, G. Vigna, C. Kruegel, and R. A. Kemmerer, “Using Generalization and Characterization Techniques in the Anomaly-based Detection of Web Attacks,” in Proc. Network and Distributed System Security Symposium (NDSS 2006), 2006.
- [24] C. Torrano-Giménez, A. Pérez-Villegas, and G. Álvarez-Marañón, “HTTP DATASET CSIC 2010,” Information Security Institute, Spanish Research National Council (CSIC), 2010.
- [25] A. Tekerek, “A Novel Architecture for Web-based Attack Detection Using Convolutional Neural Network,” Computers & Security, vol. 100, art. 102096, 2021. DOI: 10.1016/j.cose.2020.102096
- [26] M. Amouei, M. Rezvani, and M. Fateh, “RAT: Reinforcement-Learning-Driven and Adaptive Testing for Vulnerability Discovery in Web Application Firewalls,” IEEE Transactions on Dependable and Secure Computing, vol. 19, pp. 3371–3386, 2022.
- [27] C. Iliou, T. Kostoulas, T. Tsikrika, V. Katos, S. Vrochidis, and I. Kompatsiaris, “Towards a Framework for Detecting Advanced Web Bots,” in Proc. 14th Int. Conf. on Availability, Reliability and Security (ARES 2019), 2019. DOI: 10.1145/3339252.3339267
- [28] C. Iliou, T. Kostoulas, T. Tsikrika, V. Katos, S. Vrochidis, and I. Kompatsiaris, “Detection of Advanced Web Bots by Combining Web Logs with Mouse Behavioural Biometrics,” Digital Threats: Research and Practice, vol. 2, no. 3, 2021. DOI: 10.1145/3447815
- [29] R. A. Howard, “Information Value Theory,” IEEE Transactions on Systems Science and Cybernetics, vol. 2, no. 1, pp. 22–26, 1966. DOI: 10.1109/TSSC.1966.300074
- [30] C. Elkan, “The Foundations of Cost-Sensitive Learning,” in Proc. 17th Int. Joint Conf. on Artificial Intelligence (IJCAI 2001), 2001, pp. 973–978.

- [31] J. Pawlick, E. Colbert, and Q. Zhu, “Modeling and Analysis of Leaky Deception Using Signaling Games with Evidence,” *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 7, pp. 1871–1886, 2019.
- [32] S. Axelsson, “The Base-Rate Fallacy and the Difficulty of Intrusion Detection,” *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205, 2000. DOI: 10.1145/357830.357849
- [33] R. Sommer and V. Paxson, “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” in *Proc. 2010 IEEE Symposium on Security and Privacy*, 2010, pp. 305–316. DOI: 10.1109/SP.2010.25
- [34] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, “Dos and Don’ts of Machine Learning in Computer Security,” in *Proc. 31st USENIX Security Symposium*, 2022.
- [35] K. J. Ferguson-Walter, M. M. Major, C. K. Johnson, and D. H. Muhleman, “Examining the Efficacy of Decoy-based and Psychological Cyber Deception,” in *Proc. 30th USENIX Security Symposium*, 2021.
- [36] B. Schneier and J. Kelsey, “Secure Audit Logs to Support Computer Forensics,” *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, 1999. DOI: 10.1145/317087.317089
- [37] J.-H. Cho, D. P. Sharma, H. Alavizadeh, S. Yoon, N. Ben-Asher, T. J. Moore, D. S. Kim, H. Lim, and F. F. Nelson, “Toward Proactive, Adaptive Defense: A Survey on Moving Target Defense,” *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 709–745, 2020. DOI: 10.1109/COMST.2019.2963791
- [38] D. J. Schuirmann, “A Comparison of the Two One-Sided Tests Procedure and the Power Approach for Assessing the Equivalence of Average Bioavailability,” *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, 1987. DOI: 10.1007/BF01068419

In addition to the thirty-eight primary references listed above, the detection tier draws on a complementary set of recent papers covering machine-learning detection of individual web-vulnerability classes: SQL injection, file upload and webshell detection, broken access control and record-access vulnerabilities, server-side request forgery, brute force and credential stuffing, and cross-site request forgery. These are catalogued with their full bibliographic details in the project’s literature review document.